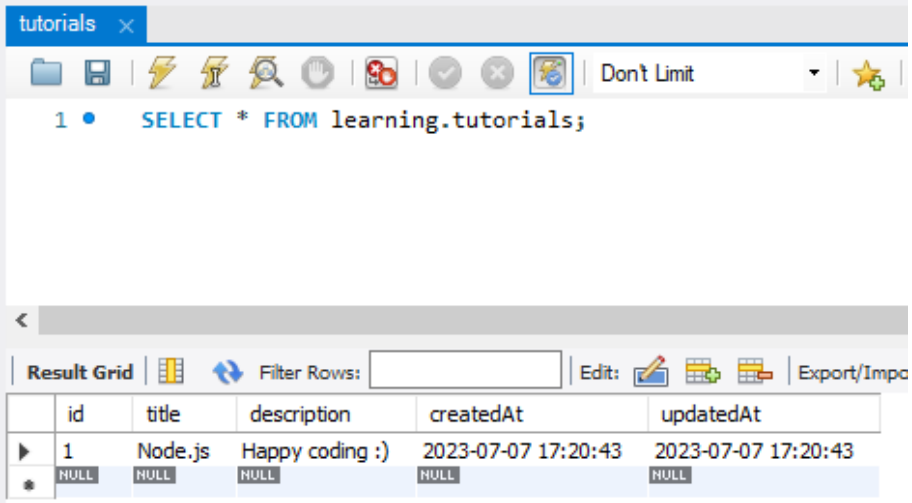
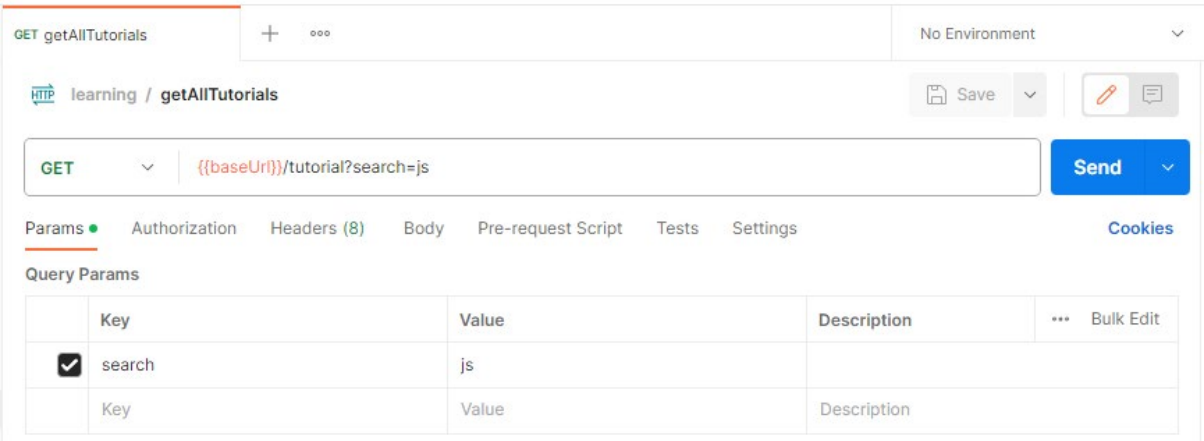
Lab 2a: Use Sequelize ORM with MySQL Database

Learning Outcomes

- Use MySQL Workbench to setup database
- Use Object-Relational Mapping (ORM) tool Sequelize to synchronize table
- Use Sequelize for Post method
- Use Sequelize for Get method





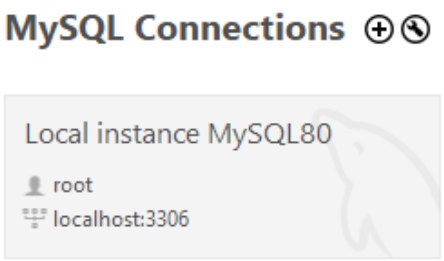
Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Software Required

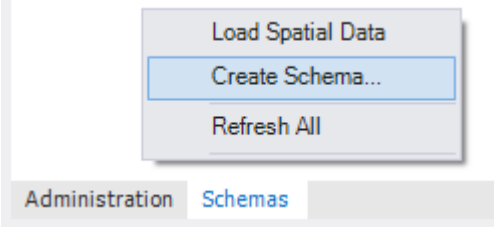
MySQL Community Server (**v8.0**) https://dev.mysql.com/downloads/mysql
Install MySQL Server and MySQL Workbench.

Activity 1: Use MySQL Workbench to setup database

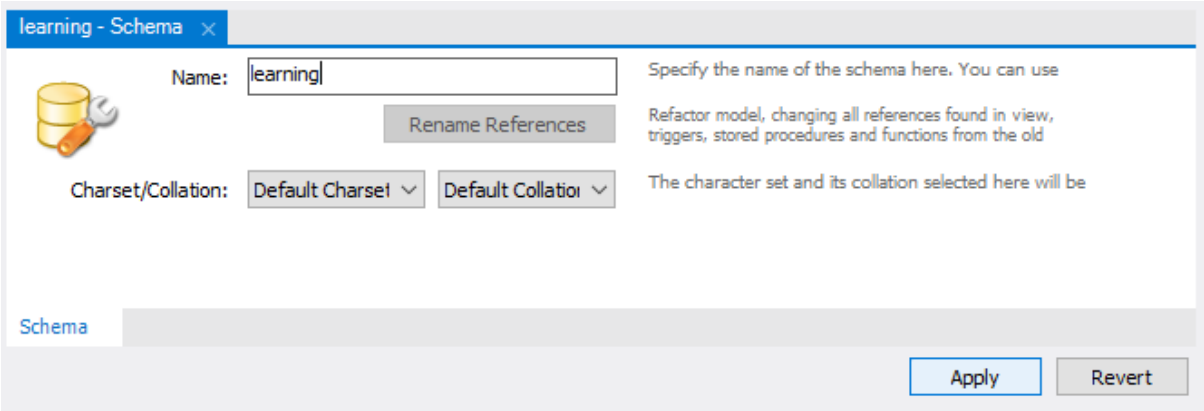
1. Open MySQL Workbench. Click the default connection using root user.



2. Under Navigator, select **Schemas**. Right click on empty space, select Create Schema. In MySQL, schema is synonymous with database.

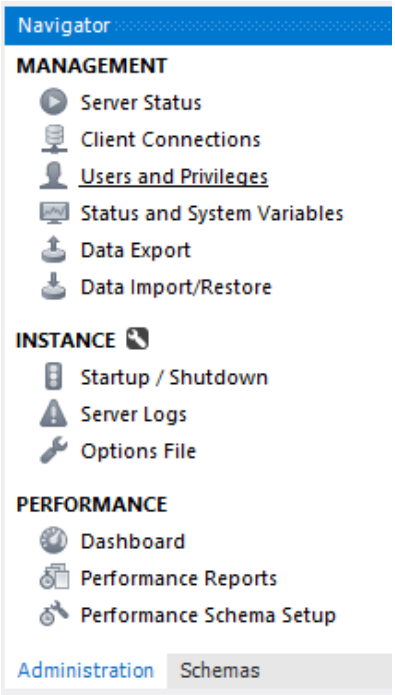


3. Enter the name **learning**, click the button Apply.

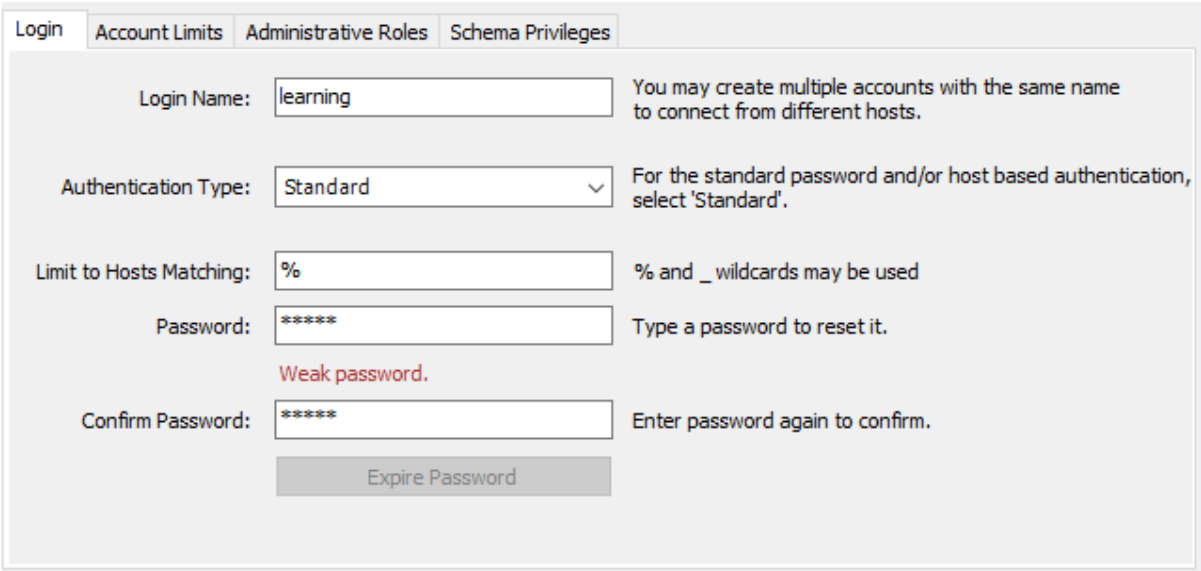


Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

4. Under Navigator, select **Administration**. Select Users and Privileges, click button Add Account.

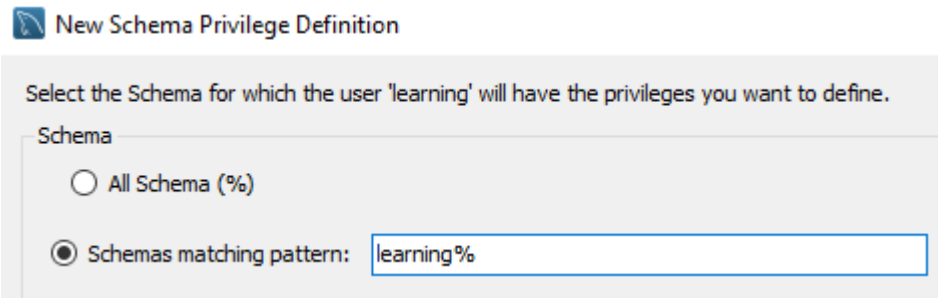


5. In account details, under Login, enter **learning** for Login Name, **mysql** for Password and confirm password.



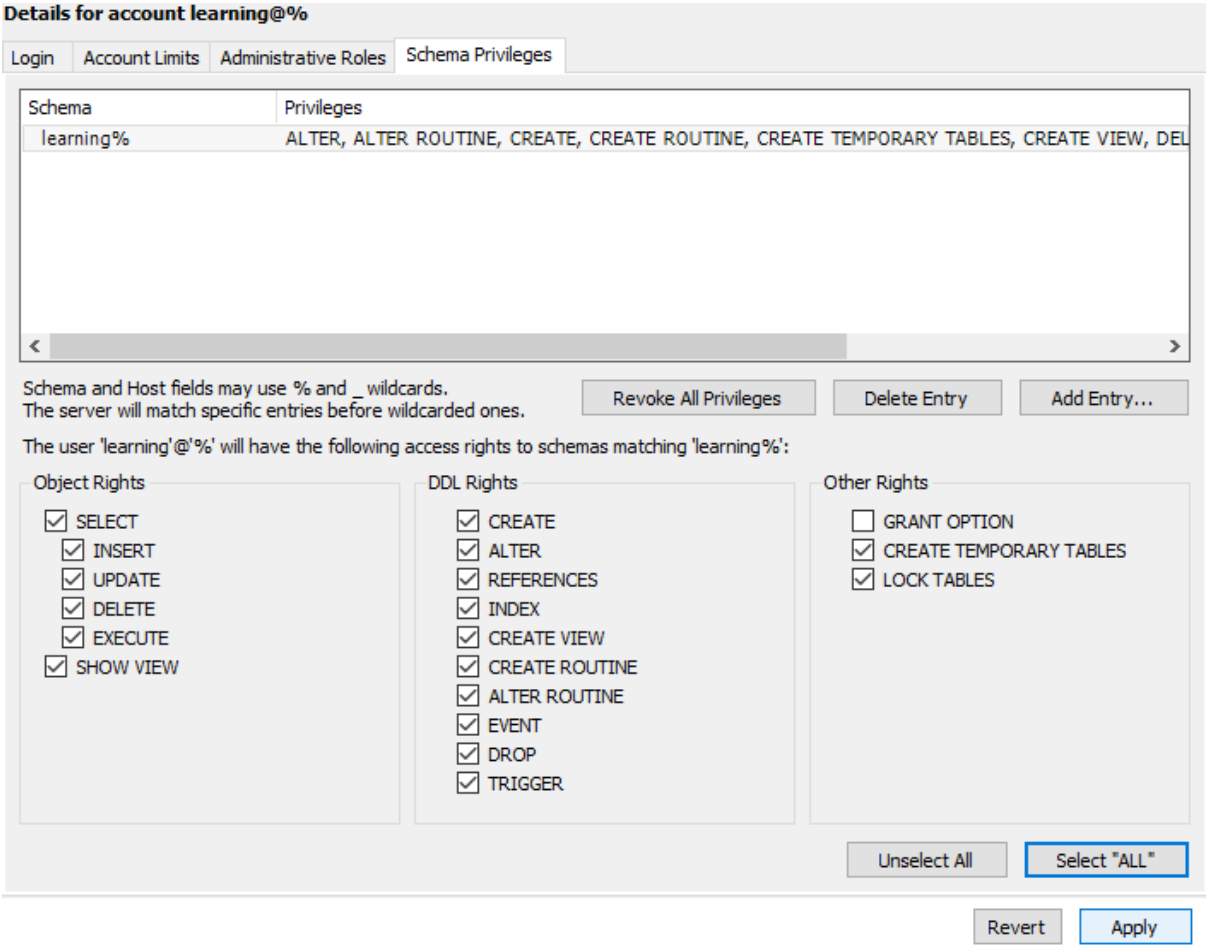
Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

6. Under Schema Privileges, click the button Add Entry. Select schema matching pattern: **learning%** and click OK.



This rule will apply to the schemas name starting with learning. You may have schemas learning_dev for development and learning_prod for production. In this practical, we only use schema name learning.

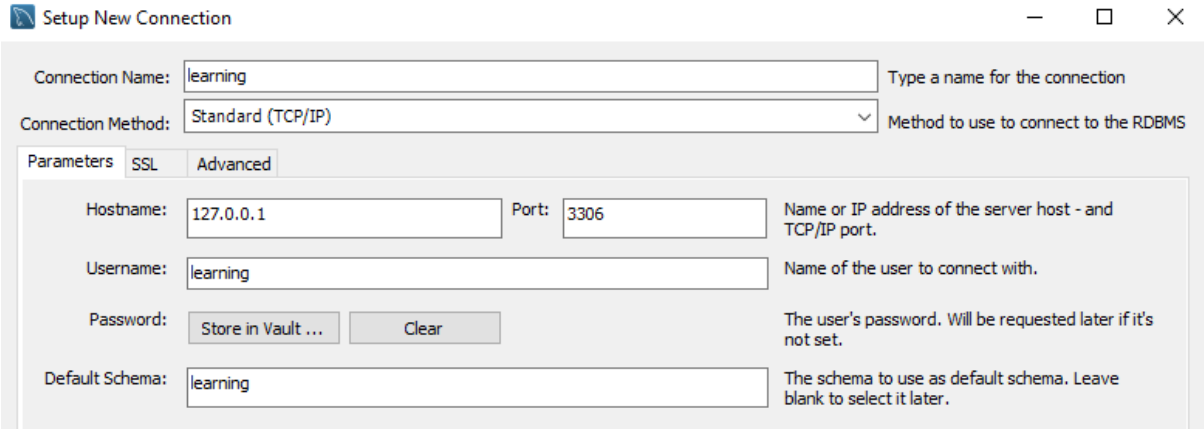
7. Click the button **Select "ALL"** to add all privileges for the selected schema. Click the button Apply.



Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

8. Close the current connection. In the home page, click the plus icon to add a new connection.

9. Enter **learning** for Connection Name, Username and Default Schema. Click the OK button to open the connection.



10. Observe that under Schemas, only **learning** can be seen and there are no tables now.

Activity 2: Use Object-Relational Mapping (ORM) tool Sequelize to synchronize table

1. Use VS Code to open the folder **learning-app**.

2. Open new terminal and run command to change directory to the server folder:
   cd server

3. Run command to install package sequelize and mysql2
   npm i sequelize mysql2
   Sequelize is an easy-to-use and promise-based Object-Relational Mapping (ORM) tool for MySQL and other databases.
   MySQL2 is a MySQL client for Node.js with focus on performance.

4. In **.env**, add database parameters:

```
DB_HOST = "localhost"
DB_PORT = 3306
DB_USER = "learning"
DB_PWD = "mysql"
DB_NAME = "learning"
```

The parameters will be used for database connection in the web server.

5. Under the folder server, add a new folder "**models**"

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

6. Under the folder models, add a new file **index.js**

```
'use strict';

const fs = require('fs');
const path = require('path');
const Sequelize = require('sequelize');
const process = require('process');
const basename = path.basename(__filename);
const db = {};
require('dotenv').config();

// Create sequelize instance using config
let sequelize = new Sequelize(
  process.env.DB_NAME, process.env.DB_USER, process.env.DB_PWD,
  {
      host: process.env.DB_HOST,
      port: process.env.DB_PORT,
      dialect: 'mysql',
      logging: false,
      timezone: '+08:00'
  }
);

fs
  .readdirSync(__dirname)
  .filter(file => {
    return (file.indexOf('.') !== 0) && (file !== basename) &&
    (file.slice(-3) === '.js');
  })
  .forEach(file => {
    const model = require(path.join(__dirname, file))(sequelize,
    Sequelize.DataTypes);
    db[model.name] = model;
  });

Object.keys(db).forEach(modelName => {
  if (db[modelName].associate) {
    db[modelName].associate(db);
  }
});

db.sequelize = sequelize;
db.Sequelize = Sequelize;

module.exports = db;
```

This file creates the sequelize instance and reads the model files from the same directory.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

7. Under models, add a new file **Tutorial.js**:

```
module.exports = (sequelize, DataTypes) => {
    const Tutorial = sequelize.define("Tutorial", {
        title: {
            type: DataTypes.STRING(100),
            allowNull: false
        },
        description: {
            type: DataTypes.TEXT,
            allowNull: false
        }
    }, {
        tableName: 'tutorials'
    });
    return Tutorial;
}
```

This data model "**Tutorial**" is mapped to the database table "**tutorials**".
The properties "**title**" and "**description**" are mapped to the database table columns.

The naming conventions:

- Model Name: in singular form and use PascalCase (each word's initial letter is capitalized). e.g. BlogPost
- Attribute Name: use camelCase (each word, except the first one, is capitalized). e.g. createdAt, userId
- Table Name: in plural form and use snake_case (lowercase with underscores between words). e.g. blog_posts

To learn more about Sequelize model, can see the link below:
https://sequelize.org/docs/v6/core-concepts/model-basics

The common used data types are as below:

	Sequelize DataType	**MySQL DataType**
Short String	STRING(max)	VARCHAR(max)
Long String	TEXT	TEXT
Boolean	BOOLEAN	TINYINT(1)
Integer	INTEGER	INTEGER
Decimal	DECIMAL(p, s)	DECIMAL(p, s)
Date Only	DATEONLY	DATE
Date Time	DATE	DATETIME

For DECIMAL(p,s): **p** is the precision that represents the total number of significant digits. **s** is the scale that that represents the number of digits after the decimal point. Example: DECIMAL(10,2) for price.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

To learn more about Sequelize data types, can see the link below:
https://sequelize.org/docs/v7/other-topics/other-data-types

8. Open **index.js** under server, change the code to start server after synchronizing the database.

```
const db = require('./models');
db.sequelize.sync({ alter: true })
    .then(() => {
        let port = process.env.APP_PORT;
        app.listen(port, () => {
            console.log(`Sever running on http://localhost:${port}`);
        });
    })
    .catch((err) => {
        console.log(err);
    });
```

The function **db.sequelize.sync()** synchronizes the data models with the database tables. This creates the table if it doesn't exist, and does nothing if it already exists.

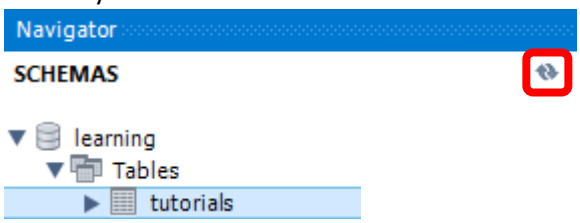
The parameter **{ alter: true }** checks what is the current state of the table in the database (which columns it has, what are their data types, etc), and then performs the necessary changes in the table to make it match the model.

Note: This parameter works well with MySQL database, but will cause issues for SQLite database with one-to-many relationships.

9. Run command to start server: **npm start**

10. Open MySQL Workbench, observe the new table **tutorials** created in the schema.

You may need to click the refresh icon to view the latest database.



Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Click the table **tutorials** to view the table information.

Information

Table: tutorials

Columns:

id	int AI PK
title	varchar(100)
description	text
createdAt	datetime
updatedAt	datetime

AI means auto increment. **PK** means primary key.
The data type **text** is used to store text without length limit.

Activity 3: Use Sequelize for Post method

1. In VS Code, open routes/tutorial.js, require data model Tutorial from the "models" folder.

```
const { Tutorial } = require('../models');
```

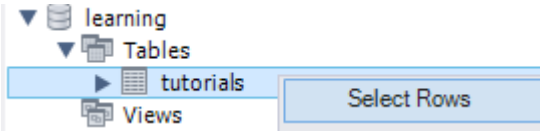
2. Change the post function to use sequelize:

```
router.post("/", async (req, res) => {
    let data = req.body;
    let result = await Tutorial.create(data);
    res.json(result);
});
```

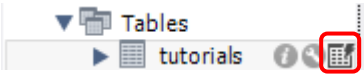
The keyword **async** is added to use together with **await**. The function now is asynchronous.
The sequelize method **create()** is used to insert data into the database table.

3. Open Postman, test the Post request to create a tutorial. Observe that the response body is the data inserted to the table.

4. Open MySQL Workbench, right click on the table tutorials to select rows.



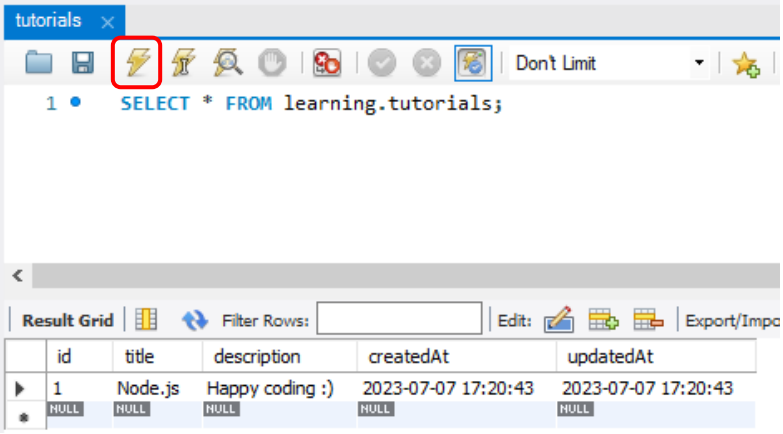
You can also mouse over the table and click on the icon to select rows.



Observe the new tutorial data is inserted to the table.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

When the SQL panel is open, you can click the Execute icon to get the latest data.



Activity 4: Use Sequelize for Get request

1. In VS Code, open routes/tutorial.js, change the get function to use sequelize:

```
router.get("/", async (req, res) => {
    let list = await Tutorial.findAll({
        order: [['createdAt', 'DESC']]
    });
    res.json(list);
});
```

The sequelize method **findAll** is used to generate a standard SELECT query which will retrieve all entries from the table.

The **order** option takes an array of items. These items are themselves in the form of [column, direction].
The direction can be '**ASC**' for ascending, '**DESC**' for descending.

2. Remove the variable tutorialList:

~~let tutorialList = [];~~

We are using the database for storage now, so this variable is no longer used.

3. In Postman, test the Get request to retrieve all tutorials. Observe that the list is from the database now.

4. Use the Post request to create more tutorials. Test Get request again to see the updated list.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

5. In VS Code, open routes/tutorial.js, import Op from sequelize.

```
const { Op } = require("sequelize");
```

The class Sequelize will be used to get the operators.

6. Change the get function to enable search:

```
router.get("/", async (req, res) => {
    let condition = {};
    let search = req.query.search;
    if (search) {
        condition[Op.or] = [
            { title: { [Op.like]: `%${search}%` } },
            { description: { [Op.like]: `%${search}%` } }
        ];
    }
    // You can add condition for other columns here
    // e.g. condition.columnName = value;

    let list = await Tutorial.findAll({
        where: condition,
        order: [['createdAt', 'DESC']]
    });
    res.json(list);
});
```

The **where** option is used to filter the query. The basic examples of the where condition are as below:

condition = { userId : 1 } => *condition: userId = 1*

condition = { userId: 1, status: 'active' } => *condition: userId = 1 and status = 'active'*

condition.title = "Node.js" => *add condition: title = "Node.js"*

There are lots of operators to use for the where clause, available as Symbols from **Op**.

The query string **search** is used to get the value from the request url.

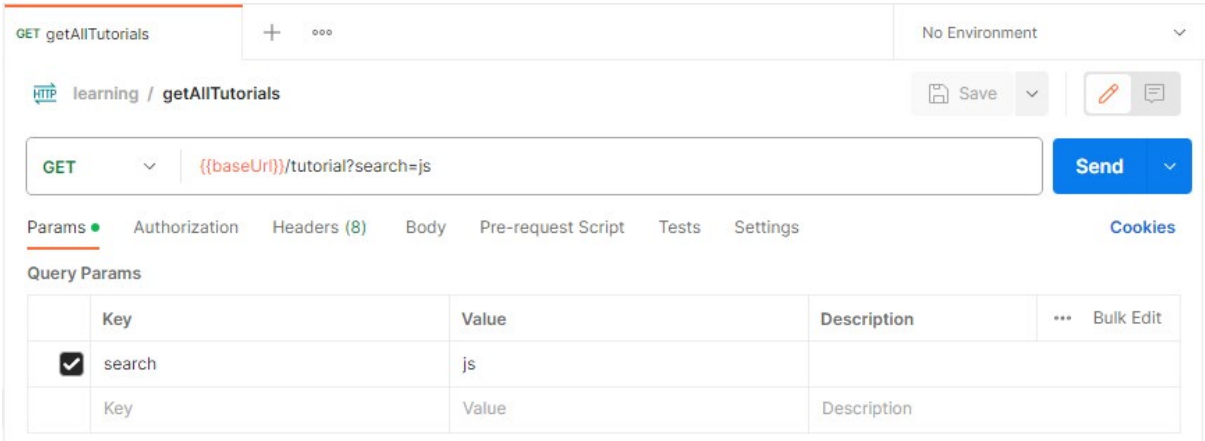
Sequelize operators or and like are used to form the condition to search title or description:
*title **like** '%<search>%' **or** description **like** '%<search>%'*

To learn more operators, you can visit the url https://sequelize.org/docs/v6/core-concepts/model-querying-basics/

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

7. In Postman, update the url of the Get request to search tutorials.

URL: {{baseUrl}}/tutorial?search=js
The query string "**?search=js**" is appended to the url. You can try other values for the search.
The query parameter key and value are also displayed under Params.



Send request and observe that the list in the response body is filtered by the search value.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.